TermTrivia: Phase 3 technical report

CIS 427 Socket Programming Project, final submission
Team: Robert and Matthew
Repository: `rfloros/CIS427_Phase2`, branch `main`
Demonstration: Thursday, November 12th, 12 to 1:45 PM.

1. Revised Curiosity Report

CIS 427 Curiosity Report (Phase 3 revision)

Project: TermTrivia, a terminal-based multiplayer trivia game
Team: Robert and Matthew
Source: `docs/curiosity-report-original.txt` (the Phase 1 report). This revision keeps the original problem, interview evidence, and user stories. It changes only what the built and tested system now shows; each change is marked *Updated* and names the test or measurement behind it.

A. Problem exploration

The problem. Groups who want to play trivia together casually, at a club meeting or a study break, do not have a good lightweight option. Platforms like Kahoot or Quizizz require an internet-connected web client, host account setup, and often a paid tier once you go past a small question limit. The low-tech alternative, one person reading questions out loud, has no real scoring, no persistent leaderboard, and frequent disputes over who answered first.

Who, when, and how it is handled today. College students and campus groups in dorm common areas, club meetings, game nights, and study breaks use Kahoot or Quizizz (account, browser, often a paid tier), verbal “shout the answer” games, or index cards. These need internet and an account, or have no scoring that cannot be disputed.

Why a networked app fits. Everyone needs to see the same question at the same time and get consistent, fair results: a classic case for a shared server pushing synchronized state to several clients.

B. Curiosity questions: what testing now tells us

1. **Who would use this most often?** Dorm and friend groups and campus clubs. Still untested with real players: whether they would choose it over Kahoot.
2. **What outcome does the user want?** Fast, fair, fun group trivia with visible scoring. Fairness (correctness-based scoring) is built in and tested: a quick correct answer and a slow correct answer both score one point (T05, and the demo with a late answer).
3. **What must be shared?** The current question, each player’s answer status, and the leaderboard. All three are held in one game object on the server and shown identically to every player in the game.
4. **What must be real time?** Question delivery and results. *Updated:* with 20 healthy localhost clients, the slowest client saw a question within 2 ms of START_ROUND, replies took 1 ms,

and the 20-second round timer closed within 6 ms of 20 seconds (`scripts/measure_nfr.sh`).

5. **What if two users answer at nearly the same time?** *Updated:* we tested it. Three clients answering at the same instant were each counted once with correct scores (T36), and two clients joining with the same name at once produced exactly one success and one 409 (T37). Each game has its own lock, so these cases are handled one at a time inside a game without slowing other games.
6. **What would make a user stop trusting the app?** A wrong or laggy leaderboard, a crash when a player disconnects, or not knowing whether an answer registered. Every command gets exactly one reply, a dropped player never crashes the server (T46), and a stalled player in one game does not delay another game (T40).
7. **What is the minimum valuable feature set?** Create or join a game by code, the host starts it, players answer, and the leaderboard updates after each round. All of it works with several clients at once.
8. **What changes from 5 players to 25?** *Updated:* we measured 20 players in one game. Broadcast time did not grow noticeably on localhost. We did not test 25 players, or players on a real network. See the scalability analysis in the technical report.
9. **Does anything persist after a disconnect?** Yes. A disconnected player can reconnect within 60 seconds and keep their score (T46). *Updated:* if the *host* does not return, a started game continues to its normal end and a lobby ends for everyone (T42, T47).
10. **What accessibility barriers exist?** A command-line interface favors people comfortable with a terminal and typing. Speed is never part of scoring, which reduces the penalty for slow typists, but a question still closes after 20 seconds.

C. User and stakeholder evidence

Source. Short interviews conducted with 2 classmates who currently play trivia in group settings.

What we learned. Both interviewees said that when they play group trivia, they typically use Kahoot. The main downside they raised was that several useful features, including creating your own games, require either payment or an account, which adds friction for a casual, one-off trivia session with friends. When we described the correctness-only scoring approach (full credit for a correct answer regardless of how fast it was submitted), both said they preferred it over speed-based scoring, since it rewards actually understanding the question rather than reaction time.

How this shaped the proposal. This evidence reinforced two core decisions: no-account, no-cost access and correctness-only scoring. Both are unchanged in the finished system. We have not run new interviews in Phase 3, so no new user evidence is claimed.

D. Target users

Primary users: hosts and players in casual trivia settings: dorm groups, campus clubs, friend hangouts. The host is also a participant and answers alongside everyone else. **Secondary stakeholders:** club organizers or RAs running a quick group activity. **Goals:** a fair, low-friction session with no account and no cost. **Likely frustrations:** input lag, uncertainty about whether an answer registered, disputes over who answered first, long gaps between rounds. **Assumptions about experience:** comfort with a terminal is likely for a CS-adjacent test group and less certain for a general dorm crowd; this remains a real limitation.

User stories. As a group organizer, I want to start a game and share a join code so that everyone can jump in quickly with no installs or accounts. As a player, I want to see my score update after

each question so that I know how I am doing. As a host, I want to see each round's results and the correct answer after it closes so that I can keep the game moving.

E. Value proposition

For casual campus groups who need a fast, fair way to play trivia together, TermTrivia provides a lightweight, terminal-based multiplayer quiz game joinable with a simple code. Unlike Kahoot or Quizizz, which require a web client, account setup, and often a paid tier, TermTrivia runs over a direct connection with no sign-up and no cost.

- **Functional value:** synchronized question delivery, fair scoring, and a live leaderboard, for several games at once.
- **Experiential value:** join in seconds with just a code; no install of anything but the client, no account.
- **Reliability value:** scoring depends only on correctness; a disconnected player can reconnect with their score and does not take down the round for everyone else (T46).
- **Scalability value:** *Updated:* the design target of about 20 simultaneous players was tested with 20 players on one machine and met the timing limits. Larger crowds and real networks remain untested.

F. Assumptions, risks, and open questions: status

1. **Assumption: terminal comfort.** Unchanged and still unverified beyond the two interviewees.
2. **Assumption: about 20 players.** *Updated:* tested with 20 on localhost.
3. **Assumption: same network.** Unchanged; all testing used localhost.
4. **Risk: a mid-game joiner starts at 0.** *Updated:* the joiner waits for the next question and sees no missed questions (T41). They still start at zero; we did not change that.
5. **Risk: telling a reconnect from a new join.** Resolved: reconnecting needs name, game code, and a private token, and every wrong guess gets the same 410 (T28).
6. **Risk: lag with about 20 clients.** *Updated:* not observed on localhost (replies 1 ms, question push 2 ms); a real network could differ.
7. **Open: how long to wait for a disconnected player?** Decided: 60 seconds by default, configurable.
8. **Open: show a mid-game joiner the missed questions?** Decided: no, they start at the next question. A join in the final intermission is also accepted and shown in the final results with score zero (T43).

2. Final problem statement and value proposition

Problem. Casual groups (dorm friends, campus clubs) lack a lightweight way to play fair, scored trivia together without accounts, payment, or a browser. Reading questions aloud has no reliable scoring or leaderboard and invites arguments about who answered first.

Solution. TermTrivia is a client-server terminal game over TCP. One server hosts many games at once. A host creates a game with a join code, players join with the code, and after START_ROUND the server runs the whole game on timers: a 20-second question, a 5-second intermission, and a 10-second final intermission. Scores come from correct answers only. A player who disconnects can reconnect within 60 seconds with a private token and keep their score.

Value proposition. For casual campus groups who need a fast, fair way to play trivia together, TermTrivia provides a lightweight, terminal-based multiplayer quiz game joinable with a simple code. Unlike Kahoot or Quizizz, it needs no account, no web client, and no payment. Its distinctive feature is **correctness-only scoring**, which two interviewed classmates preferred to speed-based scoring and which keeps the game fair for slow typists.

3. Final requirements

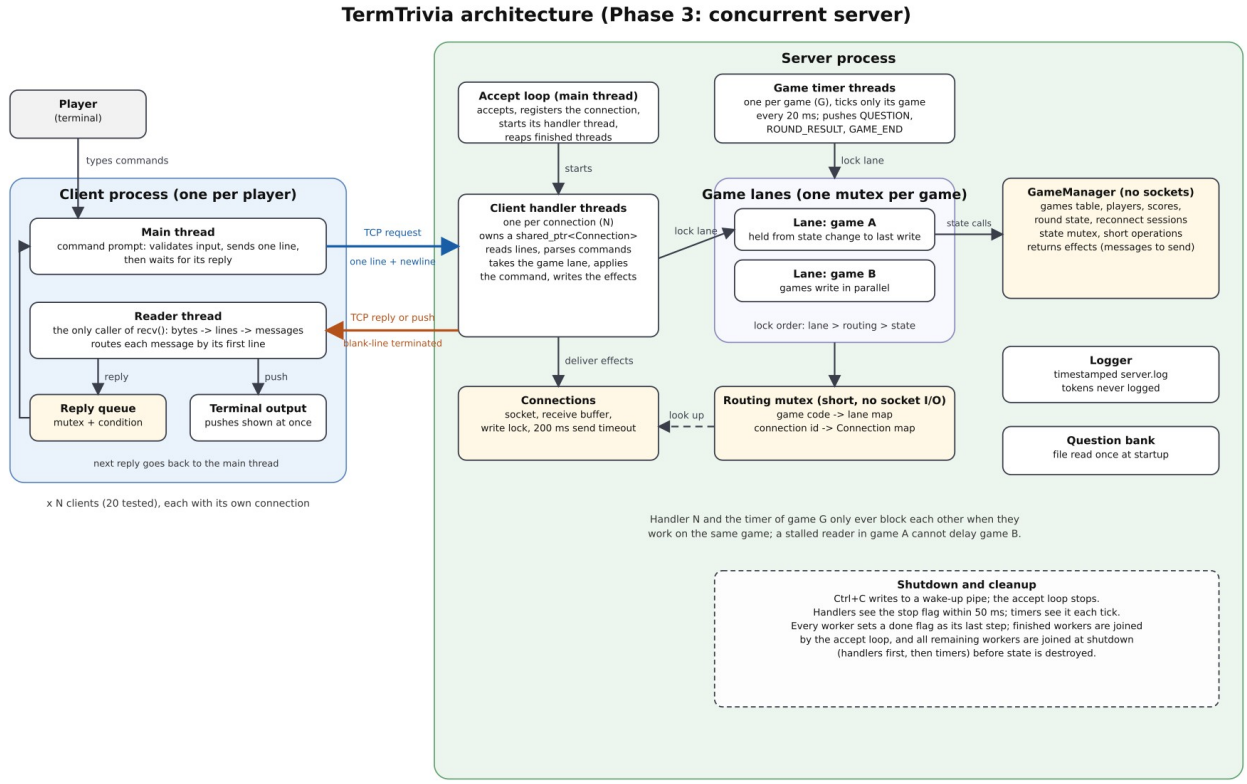
The full text, with protocol references and the tests that verify each, is in `docs/requirements.md` (26 functional and 11 nonfunctional requirements). Summary:

ID	Requirement (short)
FR-01 to FR-03	Create a game and show its join code; join by code, in the lobby, mid-game, or in the final intermission; the host starts the game once and everything after is automatic.
FR-04 to FR-09	Questions pushed to every player at once; answer A to D; scoring on correctness alone; a round closes when all active players answered or after 20 s; leaderboard on request; automatic end with GAME_END COMPLETE.
FR-10, FR-11	Host END_GAME ends only that game; QUIT is permanent, a non-host quit removes only that player and a host quit ends the game.
FR-12, FR-21, FR-22	Unexpected disconnect keeps the player for 60 s; reconnect with name, code, and token restores the score; the game does not stop when the host drops.
FR-13 to FR-20	Specific error replies (300, 401, 402, 404, 409, 410, 420, 421, 422, 423); connection stays open after a bad line; client handles bad input and server loss.
FR-23	A separate score for every player in every game; several games at once without interference.
FR-24, FR-25	Messages rebuilt from any split or combined reads; timestamped server log.
FR-26 (new in Phase 3)	Host expiry: a lobby whose host does not return ends with GAME_END HOST_EXPIRED; a started game continues to COMPLETE with no host.
NFR-01	Up to 20 players: replies within 500 ms, questions within 1 s, timers within 1 s.
NFR-02 to NFR-04	No crash on bad input or disconnects; reconnect rules; correct framing at any split.

NFR-05 to NFR-06, NFR-10	Usability, instant display of pushes, scoring independent of speed.
NFR-07	At least 3 simultaneous clients and several simultaneous games.
NFR-08, NFR-09, NFR-11	Separated modules, no tokens in output or logs, 512-byte lines, configurable ports and timers.

4. Architecture diagram

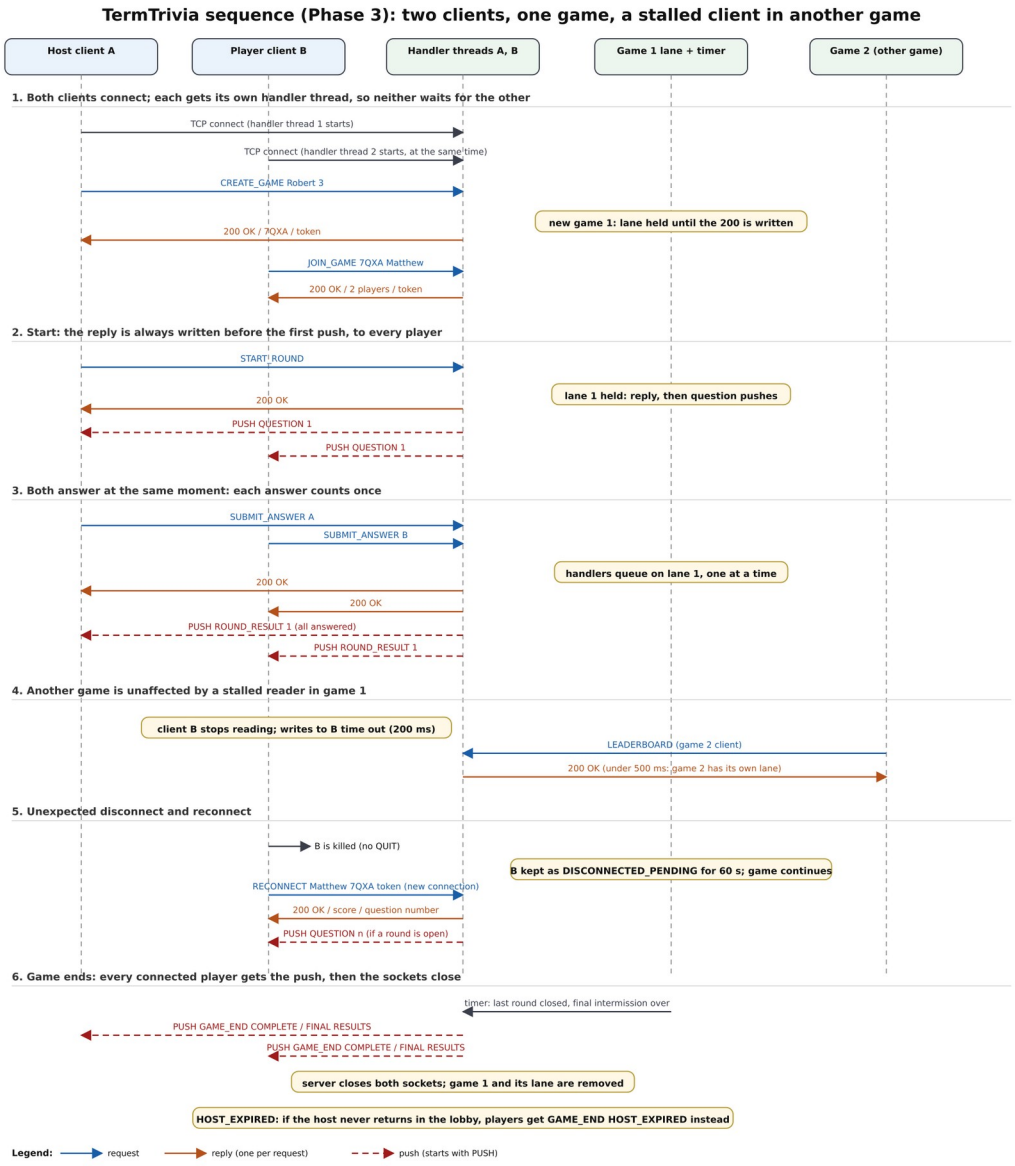
Source: `docs/images/architecture_diagram.svg` (PNG and PDF copies alongside).



Each client has a main thread (prompt), a reader thread (the only caller of recv), and a reply queue. The server has an accept loop, one handler thread per client, one timer thread per game, a lock ("lane") per game, a short routing lock, the game-state object, the logger, and the question bank.

5. Sequence diagram

Source: `docs/images/sequence_diagram.svg`.



The diagram follows two clients through connecting, creating and joining, starting, simultaneous answers, a stalled reader in another game, a disconnect and reconnect, and the end of the game.

6. Concurrency design

Why one thread per client. Each client gets its own handler thread that can simply block on its own socket, which keeps each handler short and easy for the team to read, explain, and test. The design target is about 20 players per game and a handful of games, well within what threads handle comfortably, and it follows the course's multi-threaded server sample. The cost is that threads grow with clients (section 12), which an event loop would avoid. **Sessions** are a `Connection` object per socket with a numeric client id, and a `Player` in a game holding the name, private token, score, and state (section 8). **Shared data** is the table of active games, each game's players, scores, and current question, the table of reconnectable sessions, and the server's connection and lane maps. Every command and timer tick that reads or changes them is synchronized as described below.

Threads. The accept loop starts one handler thread per connection; the handler owns that connection and is the only reader of its socket. Each game has one timer thread that ticks only that game every 20 ms. Writes can come from a handler (replies, and pushes caused by that command) or from the timer thread (question, result, and end pushes).

Per-game lane. The server first used one global lock for all games, which kept messages in order but let one slow client delay everyone. With three clients in one game not reading, a reply in another game took longer than 500 ms, and the test T40 failed on that design (5 of 6 runs). The final design gives every game a lane, a mutex that is held from the state change until all of that event's messages are written. That keeps **reply-before-push** and the order of events inside a game, while different games write at the same time.

Lock order is always: the game's lane (or the connection's own lane when it is not in a game), then the short routing mutex, then the game manager's own mutex. The routing mutex protects the game-code-to-lane map and the connection map. It may cover a state change and a registry update, but never a socket call. Because every path uses the same order, threads cannot deadlock.

Choosing the lane. A command takes the lane of the game the connection is in; JOIN_GAME and RECONNECT use the code in the command. After locking, the server looks the lane up again, and retries if the game was removed or replaced, so a stale lookup never touches a newer game that reuses the code.

Creating a game. CREATE_GAME holds the new game's lane before the game is published, starts the game's timer thread, and only then writes 200 OK. If the timer thread cannot start, the game and lane are rolled back while the lane is still held, so a concurrent JOIN_GAME cannot enter a game that is about to disappear, and the connection is closed with no reply. This resource-exhaustion path is the one exception to the one-reply rule (T44).

Delivery and lifetime. Each effect is paired with a `shared_ptr` to its connection while the state change is protected, then written outside the routing mutex under the connection's own write lock with a 200 ms send timeout. A connection stays alive until the last pending write finishes, so no write can reach a closed or reused descriptor.

Cleanup and shutdown. Every worker sets a done flag as its last step. The accept loop joins finished workers after each wake-up; at shutdown it stops accepting, lets handlers and timers see the stop flag (handlers poll every 50 ms), and joins handlers first and timers second. If a handler thread cannot be created, that connection is closed and the error is logged while the server keeps accepting (T44).

Host expiry. The timer thread expires reconnect windows. An expired host in a lobby ends the game with GAME_END HOST_EXPIRED; during play the host is just removed (T42, T47).

7. Protocol specification

Full text: `docs/protocol.md`. The protocol is line-oriented ASCII over TCP. A client message is one line ending in a newline. A server message is one or more lines followed by a blank line; a reply is the answer to exactly one command and a push starts with `PUSH`.

Command	Meaning and success reply
CREATE_GAME <name> <count>	200 OK, join code, private token
JOIN_GAME <code> <name>	200 OK, “N players in game CODE”, token

START_ROUND (host)	200 OK, then PUSH QUESTION 1 to every active player
SUBMIT_ANSWER <A-D>	200 OK (correctness is revealed in ROUND_RESULT)
LEADERBOARD	200 OK, then name:score lines, highest first
RECONNECT <name> <code> <token>	200 OK, “score question-number”, and the open question again
QUIT	200 OK, then the connection closes
END_GAME (host)	200 OK, then GAME_END HOST_ENDED to everyone

Pushes: `PUSH QUESTION n text` with options A to D; `PUSH ROUND_RESULT n` with the correct answer and standings; `PUSH GAME_END reason` with final results. **End reasons:** COMPLETE, HOST_ENDED, HOST_QUIT, and HOST_EXPIRED (new in Phase 3). **Error codes:** 300 format, 401 not in a game, 402 host only, 404 unknown code, 409 duplicate name, 410 reconnect refused (one generic reply for any wrong detail), 420 no open round, 421 already started, 422 already answered, 423 bad question count. Phase 3 additions are recorded in `docs/design-changes.md` (decisions 38 to 43).

Client update strategy: hybrid. Commands use request-response (each command gets exactly one reply). Game events use server push: QUESTION, ROUND_RESULT, and GAME_END are sent without a request and always start with `PUSH`, so the client can tell them from replies. The client’s reader thread is the only one that reads the socket; it shows pushes as soon as they arrive and hands replies to the prompt thread, so a push never corrupts typed input or another reply (T34, T39).

8. Data structures

Structure	Where	Purpose
`Game`	game.h	code, state (Lobby, QuestionActive, Intermission), players in join order, shuffled question order, current question number, phase end time
`Player`	game.h	name, private token, score, state (Active, Waiting, DisconnectedPending), connection id, host flag, answered flag and choice, time of disconnect
`games_`, `game_code_of_conn_`	GameManager	games by code; which game a live connection is in (both under the manager mutex)
`Effect`, `Outcome`	game.h	game logic returns messages to send (connection, lines, close-after), log events, and created or ended game codes;

		it never touches sockets
`lanes_`, `live_connections_`	Server	game code to lane; connection id to shared connection (under the routing mutex)
`Worker` lists	Server	handler and timer threads with done flags, so finished threads are joined
`Connection`, `LineBuffer`, `MessageAssembler`	common/	socket with a write lock and send timeout; bytes to lines (512-byte limit); lines to messages
`ReplyQueue`	client/	hands replies from the reader thread to the prompt thread

There is no persistent storage except the question bank file; games exist only in memory.

9. Error-handling strategy

- **Bad input:** every empty, unknown, malformed, or over-long line gets one error reply and the connection stays open. Input is data only and is never executed.
- **Wrong state:** specific codes (401, 402, 404, 409, 420 to 423) say what was wrong; no state changes.
- **Reconnect secrecy:** every wrong name, code, token, or late reconnect gets the same 410.
- **Slow or dead clients:** writes time out after 200 ms and mark the connection broken; the handler treats it as an unexpected disconnect. Resets and closes are handled the same way; SIGPIPE is ignored and sends use MSG_NOSIGNAL.
- **Disconnects:** the player is kept for the reconnect window and does not hold up a round; the server keeps running.
- **Host never returns:** a lobby ends with HOST_EXPIRED; a started game continues without a host.
- **Resource failure:** a handler thread that cannot start closes that client; a timer thread that cannot start rolls back the new game with no reply. Both are logged and the server keeps accepting.
- **Shutdown:** Ctrl+C stops accepting, ends every worker within about 50 ms, joins them, and logs one line.
- **Logging:** connections, requests with result codes, errors, game events, disconnects, and shutdown are logged; tokens never are.

10. Test results

Full detail, environment, and limitations: `docs/phase3-test-report.md`. Results were produced on one machine (Omarchy Linux, g++ 16.2.1, 8 CPUs) over localhost by the assistant that wrote the code. The restart test (T51) and the load test (T52) were added later and run on a cloud VM (Ubuntu 24.04, g++ 13.3.0, 2 CPUs) over localhost. On 2026-11-09 Robert re-ran `make clean && make test` on his own Linux machine, including T51, and it finished with all checks passed. The team has not re-run the 20-player measurement, the default-timer run, or the load test.

Measure	Result
Unit tests (`make test`)	116 tests run, 0 check failures
Real-socket tests	75 tests run, 0 check failures
Scripted scenarios and deliverable audit	all checks passed
Real 20/5/10/60 second timers	all checks passed (`make test-default-timers`)
Restart behavior (T51)	18 of 18 checks passed: a clean stop and a crash, each followed by a restart on the same port
Load test (T52)	55 clients: 11 000 of 11 000 requests succeeded; 210 clients: 21 000 of 21 000 succeeded (table below)
New Phase 3 socket tests (T10, T11, T22, T23, T35 to T47)	all pass; each repeated 10 times with 150 of 150 runs passing
20 players, real timers (NFR-01)	reply 1 ms (limit 500 ms), question push 2 ms (limit 1 s), 20-second round timer 20 001 to 20 006 ms (limit plus or minus 1 s)
ThreadSanitizer, 75 socket tests	no report in Phase 3 code; one report in a Phase 2 startup test (not changed)

What the key tests showed. Simultaneous answers each counted once with correct scores (T36); a same-name join race gave one 200 and one 409 (T37); ending game A left game B running (T38); three stalled readers in one game did not delay another game's replies beyond 500 ms (T40), while the earlier single-lock design failed that test in 5 of 6 runs; a lobby with an absent host ended with `HOST_EXPIRED` and a started game ran on to `COMPLETE` (T42, T47); a join in the final intermission appeared in the final results (T43); and failed thread launches, shutdown, and thread cleanup behaved as designed (T44). Testing also found one real bug, a reply that could be written after `GAME_END`; it was fixed and covered by a regression test.

Assignment test list and where each is covered

Required test (section 7.6)	Our tests
1. Three clients connecting simultaneously	T45 (three idle connections, then a fourth), T36, T52 (55 and 210 clients)
2. Three clients performing different operations	T41 (join, answer, QUIT, END_GAME in one game), T11

3. Two clients modifying shared data	T36 (three clients answer at once; every answer counted once)
4. Two clients attempting a conflicting operation	T37 (two clients join with the same name at once: one 200, one 409)
5. One client disconnecting while others remain active	T46 (killed player reconnects while others play), T26, T47
6. One client sending an invalid request while others continue	T52 (every tenth request is invalid, from all clients at once), T15
7. Repeated connections and disconnections	T44 (12 game cycles and 12 quick connections; threads cleaned up)
8. Rapid submission of multiple valid messages	T39 (300 LEADERBOARD requests during pushes), T52
9. Partial-message delivery	T31 (command split across sends), T33 (one byte at a time), T34 (client side)
10. Multiple messages arriving together	T32 (two commands in one send), T34 (reply and push in one chunk)
Unexpected client termination	T26, T46
Server handling of malformed data	T14, T15, T16, T20, T21
Request for missing or nonexistent data	T17 (unknown join code gets 404), T28 (unknown reconnect gets 410)
Invalid state transition	T19 (answer with no open round, second START_ROUND, second answer)
Graceful shutdown	T44, SIGINT scenario in tests/scripts/scenarios.sh, T52
Restart behavior	T51
Load or stress test	T52

Restart behavior (T51). With a game running, the server was stopped (once with SIGINT and once with SIGKILL) and started again on the same port. Both players' connections were closed. The old host and player tokens then got 410, the old join code got 404, and a new game could be created at once. Games live only in memory, so a restart ends every game; this is the documented behavior (section 11).

Load test (T52). Every client joins a game, then sends its requests one at a time and waits for each reply. Nine in ten requests are LEADERBOARD and one in ten is an invalid line, which must get a 300 reply without disturbing anyone else. A request counts as failed if its reply is wrong or takes more than 5 seconds. The games stay in the lobby, so no timed pushes arrive between a request and its reply.

Measure	5 games, run 1	5 games, run 2	10 games
Clients (a host and 10 or 20 players per game)	55	55	210
Requests per client	200	200	100
Requests sent	11 000	11 000	21 000
Successful requests	11 000	11 000	21 000
Failed requests	0	0	0
Test duration	9.9 s	9.4 s	22.5 s
Response time, average / median	36.4 / 35.2 ms	35.5 / 34.6 ms	154.5 / 139.9 ms
Response time, 95th percentile / slowest	60.3 / 741.0 ms	57.8 / 234.2 ms	326.0 / 1 207.6 ms
Server threads / memory at peak	61 / 5 MiB	61 / 5 MiB	221 / 8 MiB

What it showed. No request failed, the server never crashed, and it stopped cleanly with SIGINT afterwards. Threads equal the clients plus the games plus one, as designed. Response times rose from about 35 ms to about 155 ms and throughput stopped growing at roughly 1 000 requests per second. The test clients are bash processes sharing the same 2 CPUs as the server, so these times include client overhead and are best read as a comparison between runs. The slowest reply in the 210-client run took 1.2 s, above the 500 ms limit in NFR-01; that limit covers up to 20 players on a lightly loaded machine, so it is a sign of where thread-per-client starts to slow down, not a failed requirement.

Limits of this evidence: one machine, localhost only, timed play with at most 20 players in one game, a load test that stays in the lobby and uses a bash client on 2 CPUs, and timing bounds that assume a lightly loaded machine.

Phase 1 and Phase 2 test cases (T01 to T39), final results

These are the Phase 1 test plan and the Phase 2 results, rerun as part of `make test`. The full text of each test is in `docs/test-plan.md`, and the Phase 2 run is in `docs/phase2-test-report.md`. T36, T37, and T39 needed several clients and were first run in Phase 3. T10, T11, T22, T23, and T38 passed at

game-logic level in Phase 2 and with sockets in Phase 3.

Test ID	Requirement tested	Final result
T01	FR-01 Create a game	Pass
T02	FR-03, FR-04, NFR-01 Start the game and receive the first question	Pass
T03	FR-05, FR-07 Submit an answer and close the round early	Pass
T04	FR-07, NFR-01 Round closes on the timer	Pass
T05	FR-06, NFR-10 Scoring ignores speed	Pass
T06	FR-08 View the leaderboard	Pass
T07	FR-09 Game ends automatically	Pass
T08	FR-10 Host ends the game early	Pass
T09	FR-11 Host quits	Pass
T10	FR-11 Non-host quits (Phase 3)	Pass (logic and sockets)
T11	FR-02 Join a game in the lobby (Phase 3)	Pass (logic and sockets)
T12	FR-20, NFR-05, NFR-06 Client shows commands and input formats	Pass
T13	FR-25, NFR-09 Server log contents	Pass
T14	FR-18 Empty line	Pass
T15	FR-18 Unknown command	Pass
T16	FR-14 Bad CREATE_GAME arguments	Pass
T17	FR-13 Unknown join code	Pass
T18	FR-16 Command that needs a game	Pass
T19	FR-19 Command in the wrong game state	Pass

T20	FR-18 Invalid answer choice	Pass
T21	FR-18, NFR-09 Over-length line	Pass
T22	FR-17 Non-host uses a host command (Phase 3)	Pass (logic and sockets)
T23	FR-15 Duplicate name in a game (Phase 3)	Pass (logic and sockets)
T24	FR-20 Server not running	Pass
T25	FR-20 Invalid port	Pass
T26	FR-21 Client exits unexpectedly	Pass
T27	FR-12, NFR-03 Reconnect within the window	Pass
T28	FR-12, NFR-03, NFR-09 Reconnect rejected	Pass
T29	FR-20 Server stops while the client is active	Pass
T30	FR-22 Host disconnect does not end the game	Pass
T31	FR-24, NFR-04 Command split across reads	Pass
T32	FR-24, NFR-04 Two commands in one send	Pass
T33	FR-24, NFR-04 One byte at a time	Pass
T34	FR-04, FR-24 Client reassembles split server messages	Pass
T35	FR-03, FR-24 Reply written before pushes	Pass
T36	FR-23 Simultaneous answers	Pass
T37	FR-15 Simultaneous joins with the same name	Pass
T38	FR-23 Games are isolated	Pass (logic and sockets)
T39	NFR-07, FR-24 Messages are never interleaved	Pass

11. Known limitations

- One thread per client and per game: simple, and sound for the tested scale, but not for

thousands of clients.
- A client that stops reading costs its own game up to 200 ms of waiting per write before it is dropped.
- No host transfer; no persistence; all games are lost if the server stops.
- No encryption and no user accounts; reconnect safety rests on a four-character private token. All testing used localhost and one machine, never a real network. Timed games were tested with up to 20 players; the load test reached 210 clients, but only in the lobby.
- A terminal and typing are required; a question still closes after 20 seconds.
- The client lists a join code but cannot browse games; there is no chat, team mode, or custom timer per question.

12. Scalability analysis

Design. The server uses one thread per connected client and one timer thread per active game. Per client the cost is a thread stack, a socket, and small buffers; per game it is a thread, a lane, and a few hundred bytes of state. Within a game all events are handled one at a time under that game's lane; different games run in parallel.

Measured. With 20 healthy localhost clients in one game and the real timers, the leaderboard reply took 1 ms, the START_ROUND reply 1 ms, the slowest client saw question 1 within 2 ms, and the 20-second round timer closed within 6 ms of 20 seconds. Three clients that stopped reading in one game did not delay another game's reply beyond 500 ms (T40). These meet NFR-01 for up to 20 players on one machine.

Where it would stop scaling. (1) Threads and memory grow with clients and games, so a few thousand connections would be limited by threads, stacks, and context switching. (2) A broadcast writes to each player in turn inside the game's lane; with many players or slow readers one game's round could be delayed by up to 200 ms per stalled reader. (3) Everything lives in one process on one machine, with no persistence or failover. (4) The game-state mutex is shared by all games, but it is held only for short in-memory operations, never for socket I/O.

Expected behavior with 10 and 100 clients. With 10 clients the server is well inside what was measured: replies took about 1 ms with 20 clients. With 100 clients spread over several games, the load test at 55 and 210 clients suggests it keeps working with no failed requests, at the cost of about 100 threads and slower replies under load. 100 players in one timed game was not tested: each broadcast is written to the players one after another inside that game’s lane, so pushes take longer as a game grows, and one stalled reader adds up to 200 ms.

Memory and threads. Peak resident memory was 5 MiB with 55 clients and 8 MiB with 210, about 20 KiB more per added client. Each thread also reserves a stack in virtual memory (8 MiB by default on Linux), so the thread count, not memory in use, is the first limit: one per client, one per game, plus the accept loop.

Logging, contention, and bandwidth. Every request writes one log line under a single logger mutex and flushes it, so at about 1 000 requests per second the logger becomes a point where all threads wait; a buffered or background logger would fix that. Shared-data contention is kept low because each game has its own lane and the shared routing and game-manager mutexes are held only for short in-memory work, never during socket I/O. Network bandwidth is small: a question push is a few hundred bytes and a result push is about 20 bytes per player, every 25 seconds.

Storage and single-server risk. Only the question bank is stored; games are in memory, so a crash or restart ends every game (T51). There is one server process on one machine, so it is a single point of failure.

What would come next. An event loop or a worker pool, a per-connection outbound queue with a writer so a slow reader never delays a broadcast, and sharding games across processes. A load test (section 10, T52) held 55 and then 210 clients in 5 and 10 games in the lobby with no failed request; response times rose from about 35 ms to about 155 ms on a 2-CPU machine, and threads grew one per client. We did not measure 25 or more players in one timed game or a real network, so no claim is made about them.

13. Team contribution summary

Who did what

Phase	Robert	Matthew
1	Curiosity report, project proposal, architecture diagram, protocol specification, data design	Functional and non-functional requirements, testing plan
2	Set up the AGENTS.md workflow and used AI coding agents to develop, write, and test the code	Tested the server with a single client (did most of the manual testing)
3	Used the same workflow to develop and test the multi-client code	Tested the multiple-client behavior manually
Code review and demo preparation	Reviews and explains the server code	Reviews and explains the client code

The Phase 3 technical report, diagrams, and first drafts of the documentation were generated by the AI agents from the team's specifications. The team wrote this section itself.

Robert: individual reflection

My contributions. In Phase 1 I wrote the curiosity report, the project proposal, the architecture, the protocol specification, and the data design. In Phases 2 and 3 I set up the agent workflow and directed the agents that wrote and tested the code. I am responsible for understanding and explaining the server code in the demonstration.

Most important networking concept. Handling a client that disconnects. I learned how a private token ties a player to their seat, so when the player reconnects they get back their own score and nobody else can take their place.

Technical challenge. Deciding what should happen when a player drops. Should they be removed completely, or can they come back, and how do we stop someone else from taking the seat? The two problems were linked. We settled on a private token plus a reconnect window: a dropped player keeps their place for 60 seconds by default, and a connection without the token cannot claim it.

Skill to improve. My understanding of socket programming. I had not done it before this project. I used the agents to build the code, then reviewed it myself and used AI to explain what it does. I learned a lot, and there is a lot more to learn.

How my work contributed to user value. I defined the core ideas the agents built from: scoring is based on correct answers and not speed, so the game is fair, and players who disconnect can rejoin without losing their place. I gave the agents these ideas and made sure the code followed them.

Matthew: individual reflection

My contributions. In Phase 1, I worked on the functional and non-functional requirements and the testing plan. In Phase 2, I performed most of the manual single-client testing. In Phase 3, I performed manual multi-client testing.

Most important networking concept. One networking concept I learned is that the server manages the game and processes requests, while the clients communicate with the server to send commands and receive information.

Technical challenge. One challenge I had to overcome was figuring out which tests would be the most effective when testing a network.

Skill to improve. One skill I would like to improve is to better understand how TCP sockets work.

How my work contributed to user value. My work improved the application for users because, by making the requirements and testing plan, as well as doing a lot of testing, I helped make sure that the application is reliable.

14. Ripple Report reflection

Part A: Effects on users

Greatest value. Two features created the most value: correctness-only scoring and reconnect. When two players know the material, they tie, and it does not matter who types faster. That also removes the typing-speed gap, since players type commands in a terminal. Reconnect fixes a problem that games like Kahoot have, where a player who gets disconnected cannot rejoin. A crash or an accidentally closed window no longer ends a player’s game.

Reliability. The decision that improved reliability most was the reconnect token with a 60-second window. A dropped player keeps their name and score, and nobody else can take their place. Closing a round as soon as every active player has answered, and giving each game its own lock, also kept games moving and kept them independent.

Hardest part of the interface. Typing commands correctly: the spelling, and the order of arguments such as name before question count. We made these mistakes ourselves while testing. The client lists the commands at the start and shows a `Usage:` line when a command is typed wrongly.

Error handling and trust. The server answers a bad request with an error and keeps the connection open, so the player can try again. One typo does not break the game. This made it feel

sturdier and more real.

A user assumption. We assumed players would prefer fairness over speed, and the people we interviewed did. Others we talked to enjoy the speed pressure, so correctness-only scoring is a plus for some players and takes away intensity for others.

Evidence the application solves the problem. In our interviews, two classmates said that speed-based scoring is annoying even when you answer correctly, and that being unable to reconnect after a disconnect is a problem. Both are addressed by the final design. The automated tests also pass (see section 10).

Part B: Effects on teammates

Protocol and parallel work. At the start of Phase 1 we met for a couple of days to agree on the big decisions: correctness-only scoring and some form of reconnect. With those settled, the details fell into place and each of us could work on our own documents without contradicting the other. We worked in a shared Google Doc, with each document in its own tab, and we sat together to ask questions.

Helpful documentation. The tabbed shared Google Doc let each of us see what the other had done. We also used AI agents to summarize what had changed, so each of us stayed up to date.

Rework. The rework we had to do came from the professor's grading comments on Phase 1, which said the protocol, requirements, and test plan needed fixes. It did not come from confusion between the two of us.

Resolving ideas. We rarely disagreed. Our main discussion was how reconnect should work so that nobody could take a disconnected player's name. We asked the agents for options, discussed them, and chose the one that fit the scope of the project.

What we would change. We would meet and talk more toward the end of the project, as we did at the start.

Part C: Effects of networking decisions

Decision 1: Reconnect token and 60-second window.

- *Decision:* Each player gets a private token when they join or create a game. To reconnect, the player must present their name, the game's join code, and the token within 60 seconds.
- *Why:* We wanted dropped players to be able to return, but we had to stop someone else from reconnecting with a disconnected player's name and taking their place.
- *Benefit:* Dropped players keep their score, and nobody can take over their seat.
- *Tradeoff:* If the host leaves, the game keeps running on its timers and ends by itself, but nobody can end it early or use the host's controls.
- *What we would change:* Add host transfer, so the host's permissions pass to another player.

Decision 2: One lock per game.

- *Decision:* Each game has its own lock, and the server runs one thread per client.
- *Why:* Several games can run on one server, and separate groups of friends should not slow each other down. A single global lock was our first design. The team's testing showed it let

slow clients in one game delay other games, and the test failed in 5 of 6 runs (T40, section 10), so we moved to one lock per game.
- *Benefit:* Games run independently of each other.
- *Tradeoff:* A client that stops reading still delays its own game, by up to 200 ms per message, and a thread per client does not scale to thousands of connections.
- *What we would change:* Give each client an outgoing message queue and move to an event loop or thread pool.

Decision 3: No persistence (games exist only in memory).

- *Decision:* Only the question bank is stored in a file. Active games and scores exist only in memory.
- *Why:* This is a class project. Saving anything else would have been a lot of extra work and was not needed for what the project was aiming to do.
- *Benefit:* The design is simpler, and there is no file handling to get wrong.
- *Tradeoff:* If the server stops or crashes, all games and scores are lost.
- *What we would change:* Save game state in a database so it survives a crash, and so players could replay the same set of questions.

Part D: Scale reflection

If this were used by hundreds or thousands of people, the first thing we would improve is the concurrency model. We would replace one thread per client with an event loop: one thread watches all the connections and handles whichever has data ready. Thousands of threads cost too much memory and switching time. The tradeoff is that each piece of work must be short, or it would hold up every client. We would pair this with a per-client outgoing queue so a slow reader cannot delay a broadcast. After that, we would add persistence, as described above.

Part E: Use of AI tools

Tools. We used Codex, Claude, and Claude Code. For coding we used the Codex and Claude Code command-line tools on an Omarchy Linux system.

How we used them. (1) For design decisions such as reconnect, we described what we wanted and asked the agents for options, then discussed and chose together. We also had the agents walk us through the documents and draft them: we explained what we wanted, the agent asked questions, and we answered until we were confident in the draft. (2) For the code, one of us used an agentic workflow with three gates, in which Claude and Codex hand off to and check each other’s work. This is how the Phase 2 and Phase 3 code was written and tested.

Productivity. The tools greatly sped up the coding. Phase 2 was done in about an hour. For the documents, they mostly helped get our spoken ideas onto paper, and did not save as much time.

Effect on learning. Our learning shifted from syntax and code toward higher-level architecture: the scope of the project, what its functions should be, how to build them, and how to test them. The cost is that we did not write the socket code ourselves, so our understanding of socket programming is shallower than if we had. We reviewed the code and used AI to explain it, and we divided it between us: Robert reviews and explains the server, and Matthew the client.